



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Análisis Numérico para la Ingeniería - CE1111

Informe Problema 2

Profesor: Juan Pablo Soto Quirós

Alumnos:

Mauro Andrés Navarro Obando

Paula Melina Porras Mora

Grupo: 1

19 febrero 2026

1. Estrategia utilizada para resolver el problema

Una matriz pentadiagonal $A \in \mathbb{R}^{m \times m}$ posee entradas no nulas únicamente en cinco diagonales: la diagonal principal y las dos diagonales inmediatamente superiores e inferiores a ella. Esto se puede expresar como

$$A = D_0 + D_{-1} + D_1 + D_{-2} + D_2,$$

donde D_k denota la matriz diagonal con desplazamiento k ($k > 0$ hacia la derecha, $k < 0$ hacia la izquierda).

1.1. Representación mediante diagonales

Dado que la mayor parte de los m^2 elementos de A son cero, la estrategia óptima consiste en **evitar construir la matriz elemento a elemento** y, en cambio, aprovechar la función `diag` de GNU Octave, que permite crear matrices diagonales a partir de vectores en una sola llamada.

Los cinco vectores que parametrizan la matriz son:

Vector	Posición en A	Tamaño
a	diagonal principal ($k = 0$)	m
b	primera subdiagonal ($k = -1$)	$m - 1$
c	primera superdiagonal ($k = +1$)	$m - 1$
d	segunda subdiagonal ($k = -2$)	$m - 2$
e	segunda superdiagonal ($k = +2$)	$m - 2$

1.2. Construcción de los vectores (instancia del problema)

Para el sistema concreto de tamaño $m = 2500$, los vectores se definen indexando desde $i = 0$:

$$\begin{aligned} a(i) &= 2(i + 1), & i &= 0, \dots, m - 1 \\ b(i) &= \frac{i + 1}{3}, & i &= 0, \dots, m - 2 \\ c(i) &= \frac{i}{3}, & i &= 0, \dots, m - 2 \\ d(i) &= \frac{i + 2}{4}, & i &= 0, \dots, m - 3 \\ e(i) &= \frac{i}{4}, & i &= 0, \dots, m - 3 \end{aligned}$$

En GNU Octave, estas definiciones se trasladan directamente a **operaciones vectorizadas**: se construye primero un vector de índices base 0 y se aplican las fórmulas elemento a elemento, sin necesidad de bucles explícitos.

1.3. Resolución del sistema lineal

Dado el vector del lado derecho $\mathbf{h}$ con $h(i) = 2i$, $i = 0, \dots, m-1$, el sistema $A\mathbf{x} = \mathbf{h}$ se resuelve mediante el **operador backslash** (`\`) de Octave, que selecciona automáticamente un algoritmo eficiente según la estructura de A (en este caso, factorización LU con pivoteo).

La calidad de la solución se cuantifica con el **error residual**:

$$\text{error} = \|A\mathbf{x} - \mathbf{h}\|_2 = \sqrt{\sum_{i=1}^m (A\mathbf{x} - \mathbf{h})_i^2}.$$

2. Comandos de GNU Octave utilizados

2.1. `diag` — construcción de matrices diagonales

`diag(v, k)` crea una matriz cuadrada cuya k -ésima diagonal (desplazada k posiciones de la principal) contiene los elementos del vector $\mathbf{v}$. Con $k = 0$ se obtiene la diagonal principal.

Listing 1: Función `pentadiagonal`

```
1 function matrix = pentadiagonal(m, a, b, c, d, e)
2     % Validaciones de tamaño
3     if m < 5
4         error("pentadiagonal: m debe ser mayor o igual a 5");
5     endif
6     if length(a) != m, error("a debe tener largo m"); endif
7     if length(b) != (m-1), error("b debe tener largo m-1"); endif
8     if length(c) != (m-1), error("c debe tener largo m-1"); endif
9     if length(d) != (m-2), error("d debe tener largo m-2"); endif
10    if length(e) != (m-2), error("e debe tener largo m-2"); endif
11
12    % Construcción mediante suma de diagonales
13    matrix = diag(a) ... % diagonal principal
14            + diag(b, -1) ... % primera subdiagonal
15            + diag(c, 1) ... % primera superdiagonal
16            + diag(d, -2) ... % segunda subdiagonal
17            + diag(e, 2); % segunda superdiagonal
18 endfunction
```

2.2. Vectorización — operaciones sobre arreglos

Octave permite aplicar operaciones aritméticas a todos los elementos de un vector simultáneamente. Esto reemplaza bucles `for` y mejora tanto la legibilidad como el rendimiento:

Listing 2: Construcción vectorizada de los parámetros

```
1 m = 2500;
2 i_a = (0:m-1); % índices para a (m elementos)
3 i_b = (0:m-2); % índices para b,c (m-1 elementos)
4 i_d = (0:m-3); % índices para d,e (m-2 elementos)
5
6 a = 2 * (i_a + 1); % a(i) = 2(i+1)
7 b = (i_b + 1) / 3; % b(i) = (i+1)/3
8 c = i_b / 3; % c(i) = i/3
9 d = (i_d + 2) / 4; % d(i) = (i+2)/4
10 e = i_d / 4; % e(i) = i/4
11
12 h = (2 * i_a)'; % h(i) = 2i (vector columna)
```

2.3. Operador `\` — resolución de sistemas lineales

El operador `A \ h` resuelve el sistema $A\mathbf{x} = \mathbf{h}$ de manera numéricamente estable, equivalente a calcular $\mathbf{x} = A^{-1}\mathbf{h}$ pero sin invertir explícitamente la matriz (lo cual sería costoso e inestable para m grande).

Listing 3: Resolución y cálculo del error

```
1 A = pentadiagonal(m, a, b, c, d, e);
2 x = A \ h; % resuelve Ax = h
3
4 error_val = norm(A*x - h, 2); % ||Ax - h||_2
5 fprintf("Error ||Ax - h||_2 = %.6e\n", error_val);
```

2.4. `norm` — norma euclidiana

`norm(v, 2)` calcula la norma-2 (euclidiana) de un vector:

$$\text{norm}(\mathbf{v}, 2) = \|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2}.$$

El segundo argumento especifica el tipo de norma; el valor 2 corresponde a la norma euclidiana solicitada por el enunciado.

2.5. fprintf — salida formateada

`fprintf` imprime texto con formato en la terminal, similar a `printf` en C. El especificador `%.6e` muestra el número en notación científica con seis decimales, apropiado para valores de error que pueden ser muy pequeños.

3. Resumen del flujo de ejecución

1. Definir $m = 2500$ y construir los cinco vectores de parámetros mediante operaciones vectorizadas.
2. Llamar a `pentadiagonal(m, a, b, c, d, e)` para obtener la matriz A usando `diag`.
3. Construir el vector $\mathbf{h}$ con $h(i) = 2i$.
4. Resolver $A\mathbf{x} = \mathbf{h}$ con el operador `\`.
5. Calcular e imprimir el error residual $\|A\mathbf{x} - \mathbf{h}\|_2$ con `norm` y `fprintf`.